BINHAN ZHAO

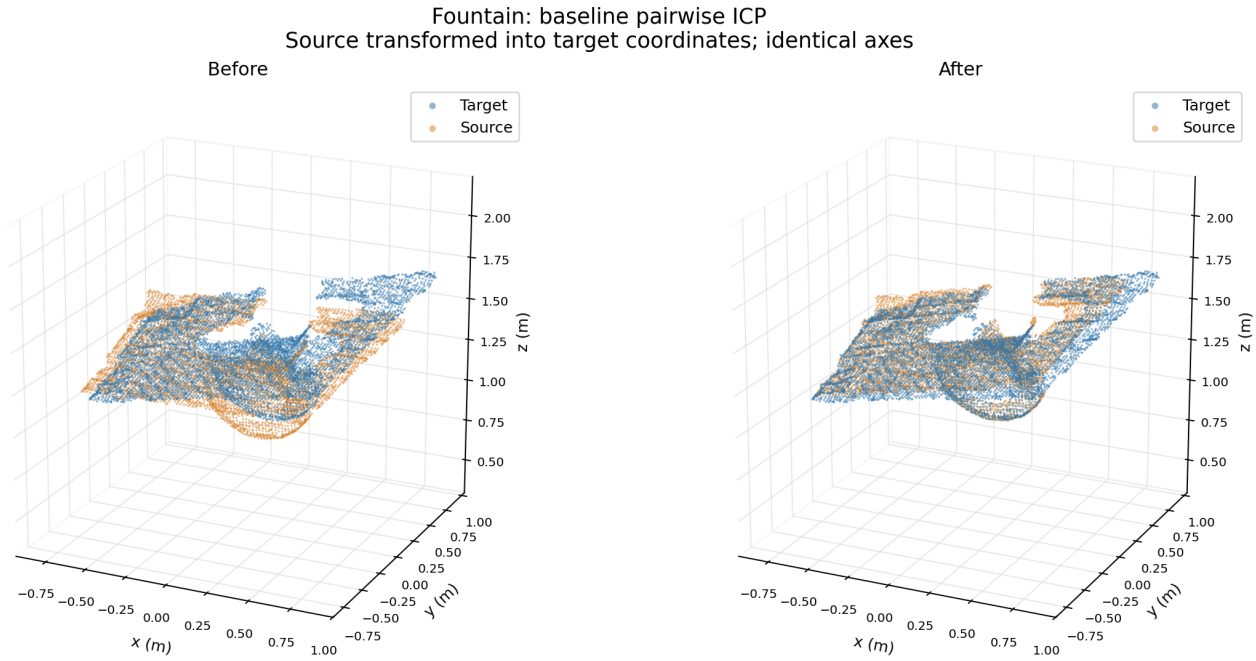
RGB-D Point-Cloud Processing and Registration

Project note | Python, NumPy, Open3D | October 2026

Question and approach

How reliably can two recorded RGB-D views be aligned, and what causes registration to fail? This offline project uses six public Fountain keyframes to examine geometry, preprocessing and pairwise registration.

The pipeline converts depth pixels to coloured 3D points using camera intrinsics, filters invalid depths, applies voxel downsampling and outlier removal, estimates normals, and calls Open3D's point-to-plane ICP solver.



Frame 0 to frame 1: the estimated transform maps source points into target-camera coordinates. Both plots use the same axes.

Recorded findings

Baseline: fixed-evaluation fitness increased from 0.2528 to 0.9658; matched-point RMSE changed from 16.76 to 6.37 mm. Evaluation uses 1 cm voxel clouds and a 3 cm correspondence threshold.

Robustness: 14 experiment records include nine preprocessing/ICP parameter combinations, poor initialization, restricted image coverage, an empty-match check and two additional frame pairs. The poor-initialization case reached only 0.0592 fitness, illustrating a failure case.

Checks: five unit tests cover back-projection, camera-coordinate conventions, a known synthetic rigid transform, agreement with Open3D projection and empty correspondences.

Reproducibility and scope

The source package includes Python modules, pinned dependencies, sample data with provenance and checksums, tests, CSV/JSON results, 4x4 transforms, PLY point clouds and an HTML experiment report. Original run: 23 October 2026. Tests were rerun successfully on 27 October 2026.

Interpretation: fitness is a source-point matching fraction, not accuracy; inlier RMSE is not pose error. The supplied trajectory is an estimated reference, not independent ground truth. This exercise does not demonstrate ROS integration, sensor calibration, live camera capture or deployment on a robot.

Data: Q.-Y. Zhou and V. Koltun, *Color Map Optimization for 3D Reconstruction with Consumer Depth Cameras* (2014), via Open3D SampleFountainRGBDImages. Dataset source | Open3D ICP documentation.

BINHAN ZHAO

Automatic Volume Control for an Audio Amplifier

EE2073 course laboratory | AY2025/26 Semester 2 | Group project, individual report

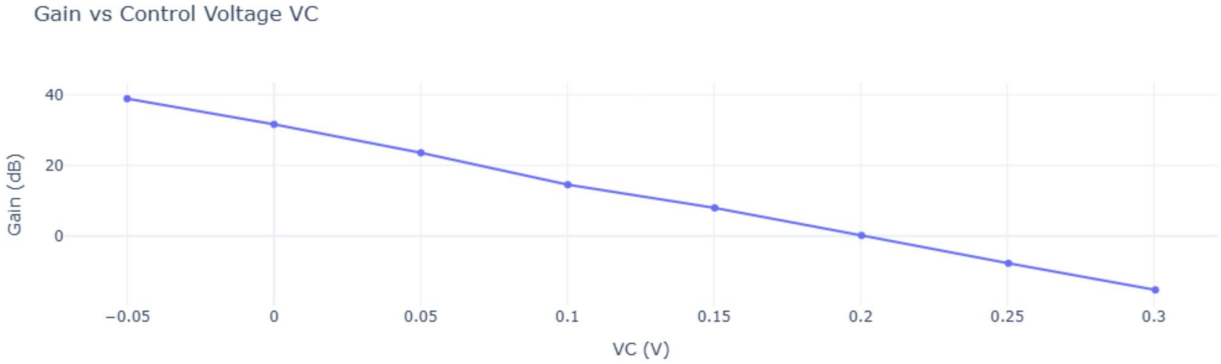
System and laboratory work

This course laboratory project combined a voltage-controlled amplifier (VCA), power amplifier and volume-unit (VU) meter into an adjustable audio amplifier. My individual EE2073 report records the subsystem measurements, integration checks and problems encountered during testing.

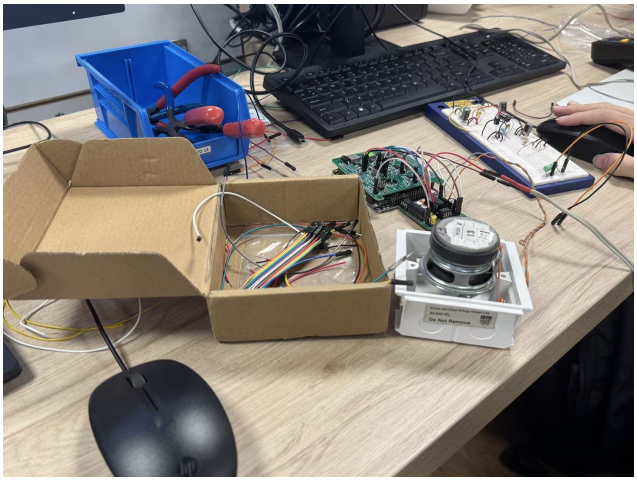
A microcontroller and Python-based computer interface support waveform generation, acquisition and parameter adjustment. The VU meter rectifies and smooths the signal into a DC-like level reading, allowing changes at the audio output to be observed during integration.

Measurements

VCA gain was calculated as 20 log10(Vout/Vin) while control voltage was varied. Lower control voltage gave higher gain; high-amplitude signals could clip. The power amplifier was tested from 100 Hz to 50 kHz at 0.1 V and 0.2 V peak inputs. Integration checks compared audio output and VU-meter readings as input level and control voltage changed.



Original report, Figure 2.4.3 (p. 18): measured gain against control voltage.



A useful debugging case

An incorrectly connected negative supply damaged an OP275 op-amp. The report records tracing the fault, correcting the connection and replacing the component before testing again. It reinforced the importance of checking power rails and individual stages before judging the whole system.

Original laboratory setup: Figure 4, p. 19.

Evidence: the original EE2073 laboratory report, sections 2.1-2.4. Figures show the original laboratory work; no new measurements were generated for this portfolio. A quantified closed-loop regulation response and standalone controller code are not included.

BINHAN ZHAO

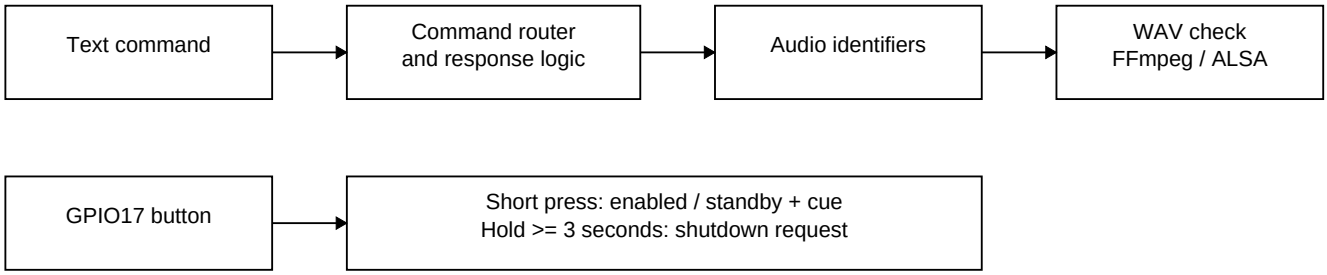
Voice-Interactive Companion Robot

Ongoing student project | July 2026 - Present | ENGenious Sparks Grant 2026

Project aim

I am developing a small companion robot as a year-long student project supported by the ENGenious Sparks Grant 2026. The aim is to connect understandable interactions with modular software, so input handling, responses and physical controls can be developed separately.

Current software structure



Module boundaries in the existing Python source.

The current Python modules separate text-command routing, response selection and audio playback. The router handles functions such as time, date, greetings and fallback responses. Functions return audio identifiers; the speaker module checks for the corresponding WAV files, combines available clips with FFmpeg and plays them through ALSA. This separation helps distinguish a missing audio file from a routing problem.

A Raspberry Pi GPIO17 button module provides a physical control. A short press toggles the enabled/standby state and selects an audio cue. A hold of at least three seconds requests the shutdown sequence. These behaviours are represented in the existing source code.

Motion and object-memory design

The motion notes describe parameters for eyes, eyelids, mouth, head and limbs, including timing and return-to-neutral behaviour. Speech-driven mouth movement is separated from expression offsets. These are actuator-integration design targets, rather than measured motion results.

My separate belief-revision notes explore object identity, last observation time, location and uncertainty as a basis for object memory. This remains design work in the available materials. The next step is to connect the interaction modules to hardware and trace one input through response selection, audio playback and movement, recording failures and timing.

Materials behind this snapshot

Existing Python modules: dialogue.py, speaker.py, button.py and date_time.py; motion-template notes; and a separate belief-revision notebook. This is a source-and-design snapshot, not a measured demonstration of an integrated speech, vision and servo system.

Public portfolio and project notes